



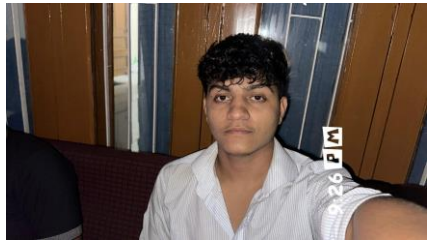
First Phase Presentation

PROJECT AND TEAM INFORMATION

Project Title

Smart Task Scheduling and Priority Management Using Data Structures

Student / Team Information

Team Name:	Avengers
Team member 1 (Team Lead)	
Name:	Captain
Student ID:	25181005
Email:	<i>captainattri102@gmail.com</i>

PROPOSAL DESCRIPTION

Motivation

Students, employees, and small teams often handle several tasks simultaneously. When tasks have different priorities and deadlines, deciding which task should be completed first becomes difficult. Simple to-do lists usually store tasks but do not schedule them efficiently. As a result, important work may be delayed, deadlines may be missed, and users may spend unnecessary time manually reorganizing their tasks.

The proposed Smart Task Scheduling and Priority Management System will solve this problem by organizing and processing tasks using fundamental data structures and algorithms. Users will be able to add, delete, display, and search for tasks. The system will also determine a suitable execution order based on task priority, deadline, and arrival time.

The project is important because it demonstrates how concepts studied in Introduction to Data Structures can solve a practical scheduling problem. A queue will support normal first-come, first-served processing, while a priority queue and heap will efficiently manage urgent tasks. Searching and sorting algorithms will help users locate tasks and arrange them by deadline or priority. A linked list may also be used for dynamic task storage.

The system will produce measurable results, including task-processing order and waiting time. It will therefore provide both a useful task-management application and a clear demonstration of the performance and practical value of different data structures

State of the Art / Current solution

Task management is currently handled through paper-based lists, spreadsheets, calendar applications, and digital tools such as Microsoft To Do, Trello, and similar task-tracking systems. These solutions allow users to create tasks, assign deadlines, and mark work as completed. However, users commonly select priorities and arrange tasks manually, while the internal scheduling process is not visible.

For an introductory data structures project, existing commercial systems are also too broad and complex. They often include collaboration, notifications, cloud storage, automation, and enterprise features that are outside the project's academic scope.

The proposed system focuses specifically on the data structures and algorithms behind task scheduling. It will demonstrate normal queue processing, priority-based scheduling, deadline sorting, task searching, and waiting-time calculation. It will also compare different scheduling approaches so that their behavior and efficiency can be understood.

Project Goals and Milestones

The main goal is to design and implement a working task scheduler that stores, organizes, searches, and processes tasks according to their priority and deadlines. The project will demonstrate the practical use of queues, priority queues, heaps, linked lists, searching, and sorting algorithms.

<i>Phase</i>	<i>Milestone</i>	<i>Expected completion</i>
<i>Phase 1</i>	<i>Implement task creation, deletion, display, searching, and a basic queue.</i>	<i>30%</i>
<i>Phase 2</i>	<i>Add priority levels, a priority queue, heap operations, deadline-based sorting and the scheduling algorithm.</i>	<i>30%</i>
<i>Phase 3</i>	<i>Integrate all components, calculate waiting time, compare performance, test the system, and complete the interface.</i>	<i>40%</i>

The initial milestone is to define the task structure and implement basic task-management operations. Each task will have information such as a unique ID, title, priority, deadline, estimated duration, and status. The basic queue will demonstrate first-in, first-out task processing.

The second milestone will introduce smart prioritization. The final milestone will produce a complete scheduler that automatically determines task order by considering priority, deadline, and arrival order.

Project Approach

The project will follow an incremental development approach. During the first phase, a task model will be created with fields such as task ID, title, priority, deadline, estimated duration, arrival time, and status. Basic operations will include adding, deleting, displaying, and searching for tasks. A queue will initially process tasks in first-in, first-out order.

In the second phase, a priority queue implemented using a heap will be introduced. High-priority tasks will be processed before lower-priority tasks. If two tasks have the same priority, their deadlines or arrival order will be used as tie-breakers. Sorting algorithms will arrange tasks by deadline or priority, while searching algorithms will locate tasks using their IDs or titles.

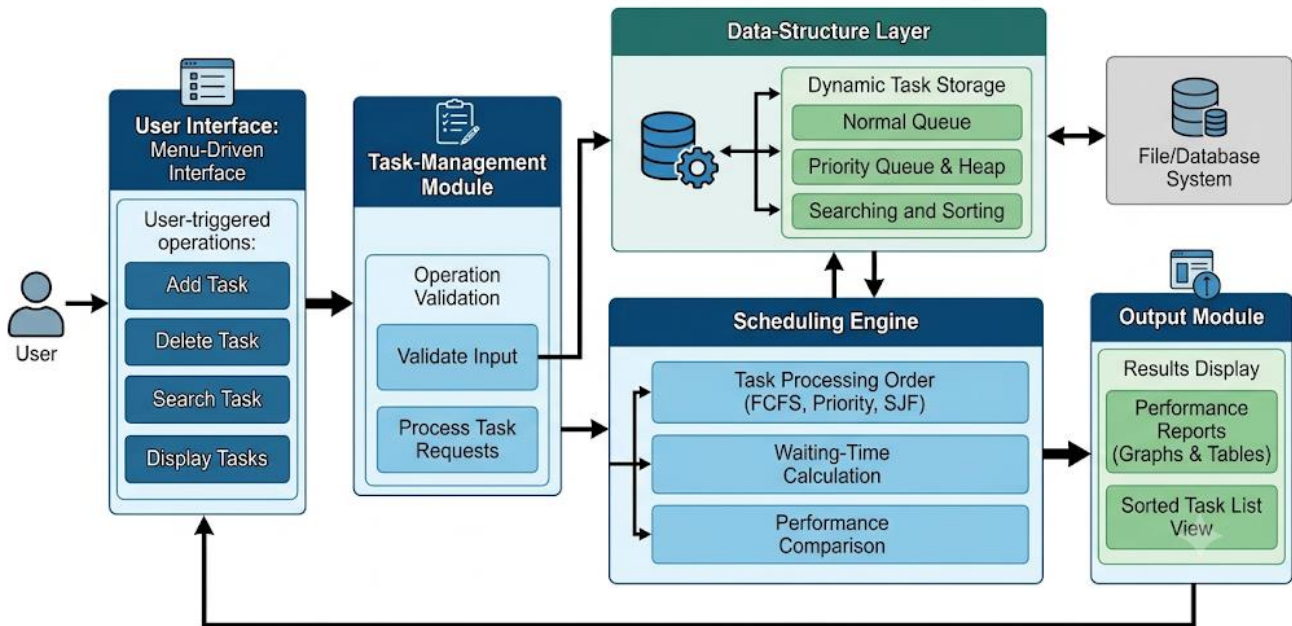
In the final phase, the components will be integrated into a complete scheduling system. The scheduler will automatically create an execution order, calculate the waiting time of each task, and compare normal queue scheduling with priority-based scheduling.

The system can be developed as a menu-driven console application using C++, with structures or classes for task records and standard data structures implemented manually where required. Testing will include normal, empty, duplicate, and boundary cases. The current system will remain focused on data structures; machine-learning-based priority prediction will be treated as a future enhancement.

System Architecture (High Level Diagram)

The system contains a user interface, task-management module, data-structure layer, scheduling engine, and output module.

Comprehensive Task Management System Architecture Diagram



Project Outcome / Deliverables

The project will deliver a functional task scheduling and priority management application based on fundamental data structures. Its main deliverables will be:

- A task record containing an ID, title, priority, deadline, estimated duration, arrival time, and status.
- Functions for adding, deleting, displaying, updating, and searching for tasks.
- A basic queue for first-in, first-out task processing.
- A priority queue implemented using a heap.
- Sorting facilities for arranging tasks by deadline and priority.
- A scheduling algorithm that handles priority, deadline, and arrival order.
- Waiting-time calculations for scheduled tasks.
- A comparison between normal queue processing and priority-based scheduling.
- Test cases, source code, system documentation, and a final working interface.

The expected outcome is a system that demonstrates how selecting an appropriate data structure improves task organization and scheduling efficiency. It will also provide a strong foundation for a future intelligent scheduling system that predicts task duration and priority.

Assumptions

Each task will have a unique ID and valid information. Priority will use predefined levels such as high, medium, and low. Deadlines and estimated durations will be entered by the user. The scheduler will assume that one task is processed at a time and that tasks are not interrupted after execution begins. When two tasks have the same priority and deadline, arrival order will determine which is processed first. Data may be stored in memory during the initial phase, with file-based persistence added later if time permits.

References

- ② Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press.
- ② Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. *Data Structures and Algorithms in C++*. Wiley.
- ② GeeksforGeeks. *Data Structures and Algorithms*. [geeksforgeeks.org](https://www.geeksforgeeks.org/)
- ② cppreference.com. *C++ Standard Library Reference*. en.cppreference.com
- ② Programiz. *Data Structures and Algorithms*. [programiz.com](https://www.programiz.com/)